



Mansoura University
Faculty of Computers and Information
Department of Computer Science



Spring 2026

[SWE143] Software Project Management

Lecture 03: Software Development Activities – Part 1

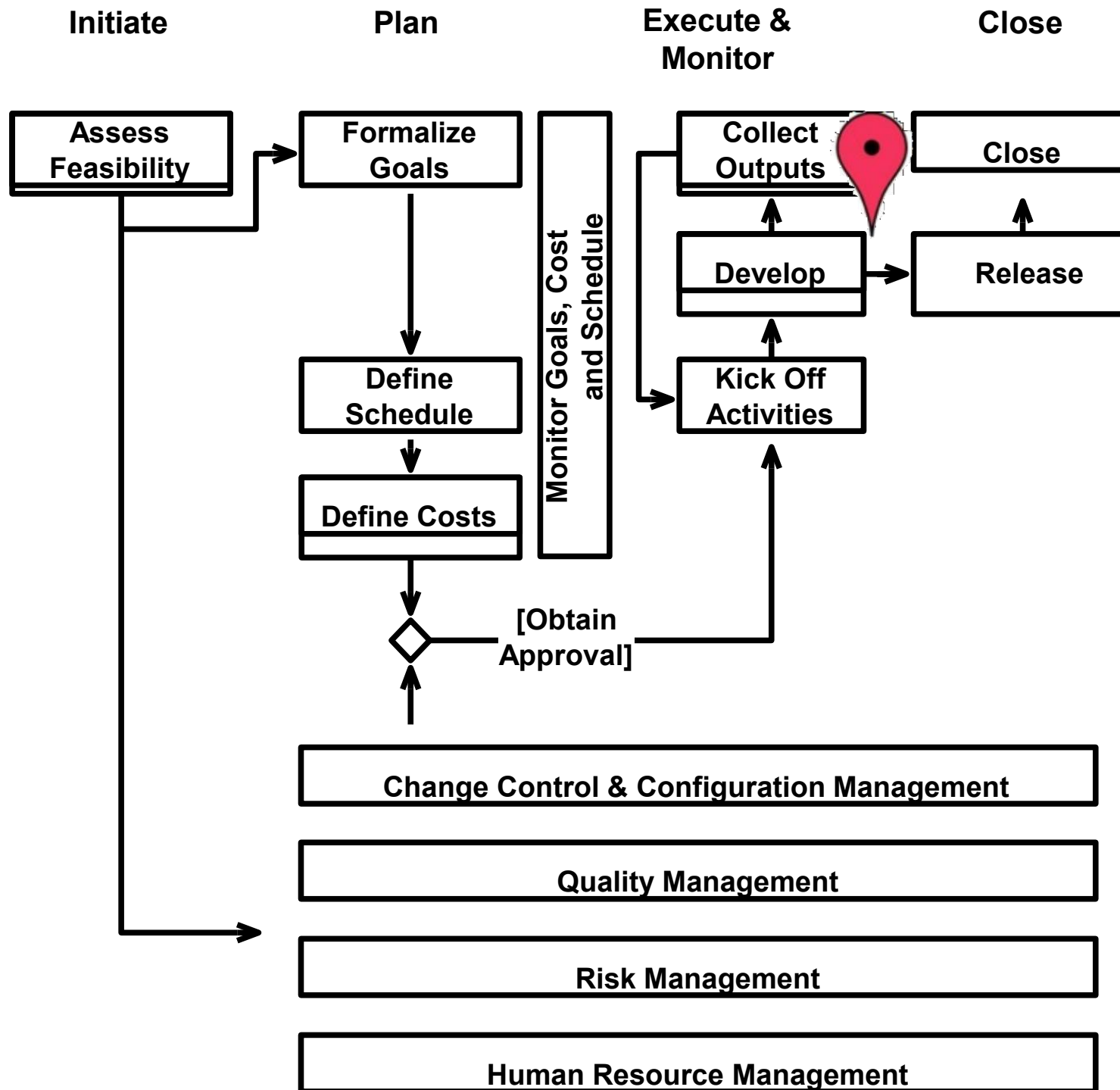
Level 300: software engineering program

Muhammad Haggag Zayyan, Ph.D

(Traditional) Software Development Processes

Goals of the Unit

- A gentle and high-level introduction to software development activities
- Understanding what are the building blocks for producing software
- Remarks:
 - This is no substitute for a software engineering course
 - The activities need to be integrated in a coherent process, to make sense
 - Software development projects range from the very small to the very large ... not all activities equally useful or relevant in any context



Overview

- Software development is a progressive تحسين تدريجي refinement which moves from concept to operations through the following phases:
 - Requirements and User Experience Design
 - Design
 - Implementation
 - Verification and Validation
 - Deployment
 - Operations and Maintenance
- As we move along these phases, we make and commit to specific choices; the cost of changes increases accordingly.
- Different processes put different emphasis on each activity or define the order in which these activities can be performed.
 - Waterfall Model (linear approach) / Agile Methodology (iterative development)

Requirements Management

Requirements Types

- Goal:
 - Forming a shared view about the characteristics of the system to build
- Output: List of requirements, presented as:
 1. Textual
 - * a text document (verbs): Functional / Nonfunctional requirements.
 - * user stories (structure): used in Agile development
 - using this pattern: As a [user] I want to do [this] because of [that].
 2. Diagrammatic
 - * a set of diagrams (e.g., use case diagrams) and corresponding textual descriptions

1. List of Requirements

- Format:
 - It is written in natural language (plain English) rather than a strict formal format like tables, diagrams, or a specification language, and it consists of free text describing the functions and other properties of a system.
- Advantages
 - Simple to draft and distribute
 - The format can be used to keep track of changes (versioning)
- Disadvantages
 - No focus on user interaction: it can be difficult to understand for a customer
 - Ambiguities and incoherencies; interactions among requirements

User Stories

- Format:

- Structured textual descriptions of user functions:

- As a [description of user],
 - I want to do [functionality]
 - So that [benefit]

defines the
who,
what and
why
of a product feature

- Advantages

- Intuitive, compact, and simple to understand for a customer
 - It focuses on what the system does (user functions)

- Disadvantages

- Difficult to represent and keep track of non-functional requirements
 - It is a partial specification (many details need to be worked out during the implementation) - used by Agile methodologies

User story examples

Agile User Story Writing Template



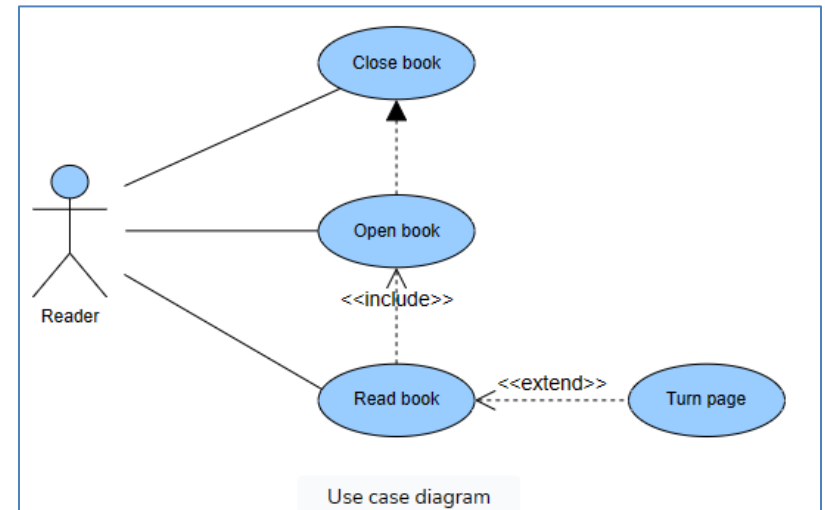
Use story no	As a <type of user/ persona>	I want to <goal/ objective>	So that <benefit/ result/ some reason>
1	Project team member	know all my tasks in advance	I can prepare and plan my time properly
2	Content manager	get a weekly report of content analytics	I can monitor the effectiveness of content writer
3	CEO	get a weekly report from all department heads on their team goals	I know whether my strategy is working
4	Middle-aged woman on a sabbatical	earn a certificate for digital marketing from coursera	I can restart my career
5	Software developer	reskill myself by attending the training program organized by my employer	I can stay employed

2. Use Case Diagrams

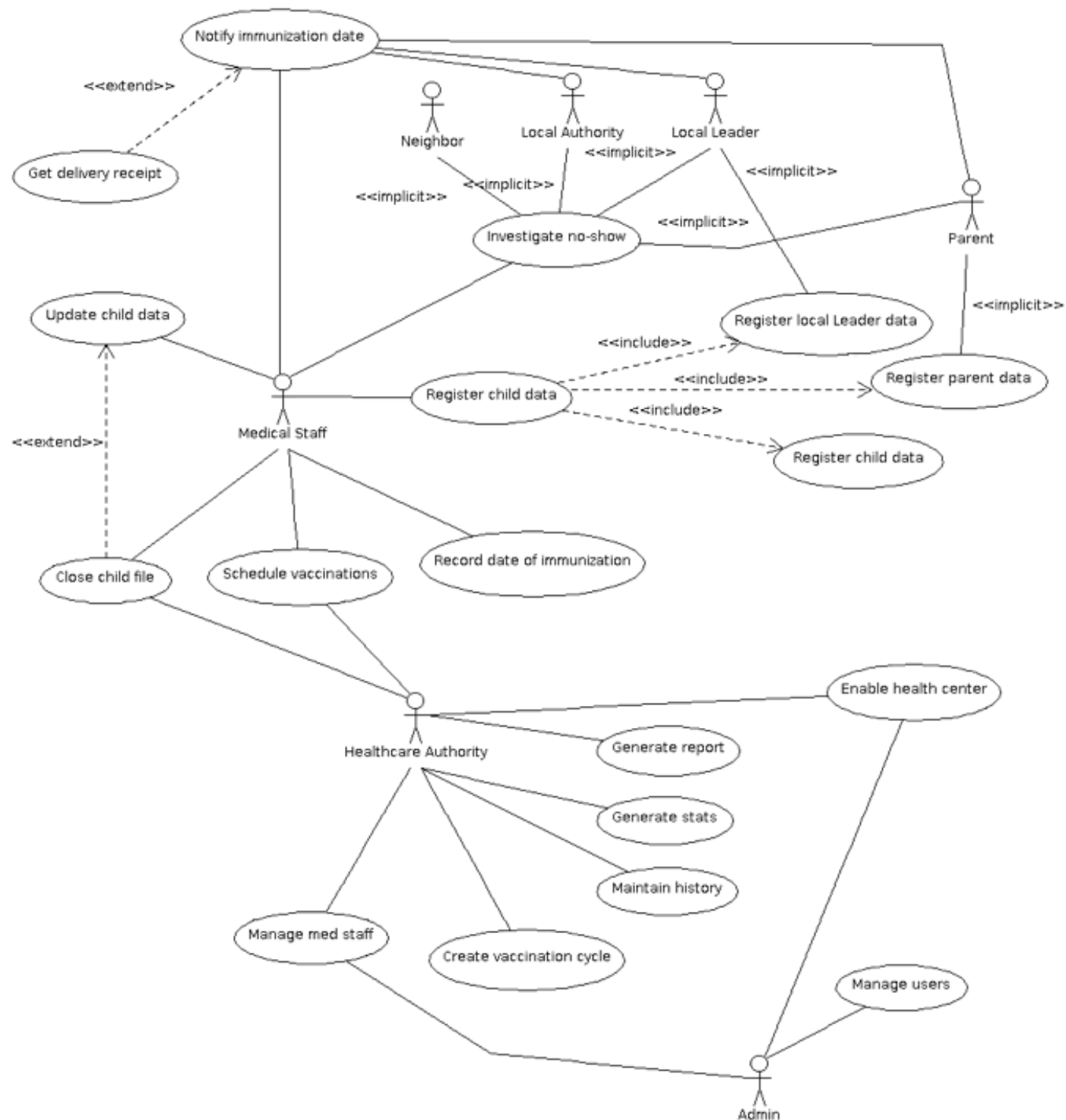
- Format:
 - Diagrams describing the interaction between users and the system
 - Textual description of the interaction as a sequence of steps
- Advantages
 - Intuitive, simpler to understand for a customer
 - It focuses on what the system does (user functions)
- Disadvantages
 - Difficult to represent and keep track of non-functional requirements
 - Managing diagrams requires a bit more work than working with text only

A UML use case diagram

- A UML use case diagram summarizes the interactions between a system and its users
- The use case diagram displays three kinds of entities:
 - the system
 - the actors,
 - the use cases.

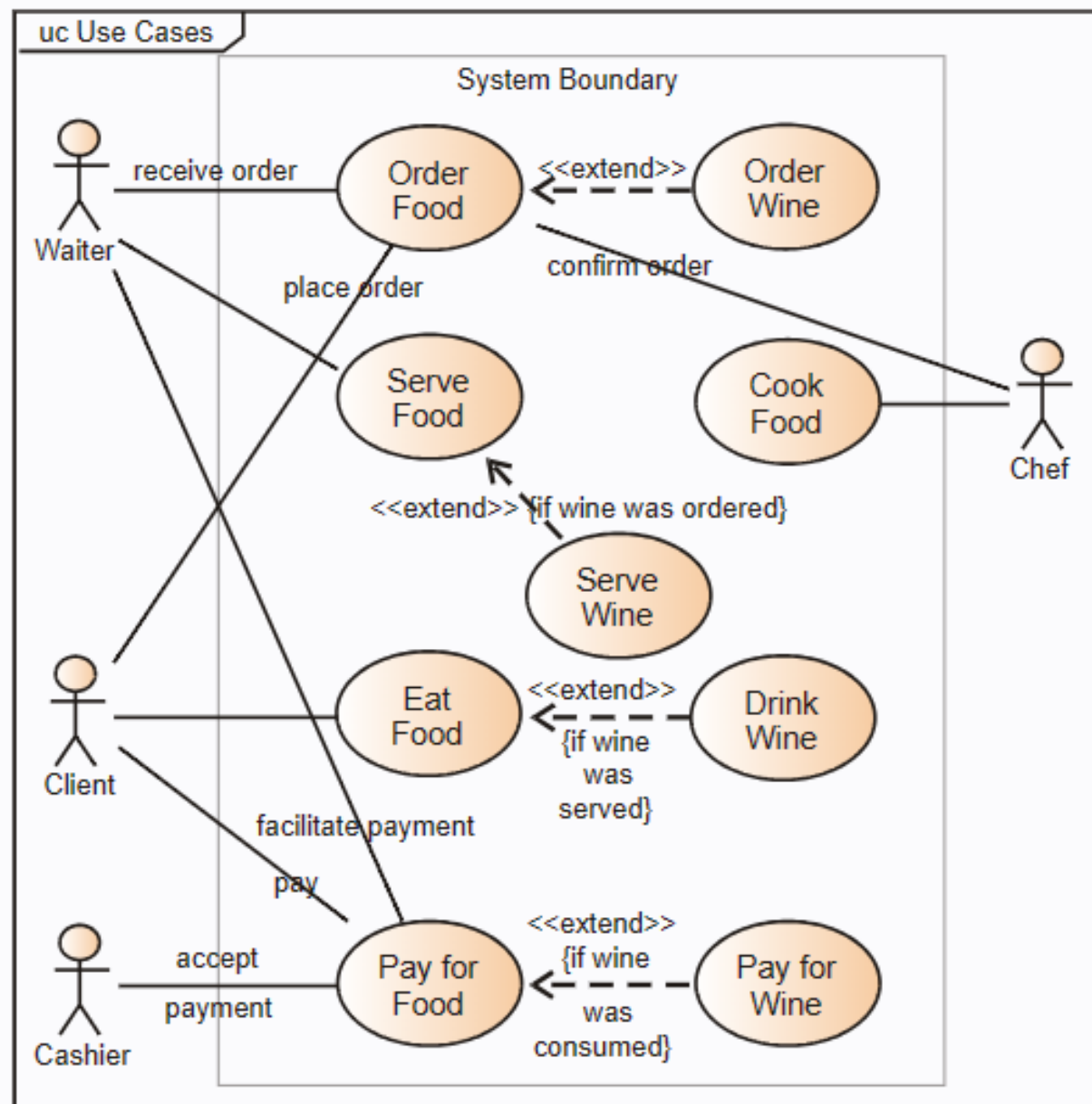


- To represent complex relationships between different use cases:
 - **Extend relationship:** The use case is optional and comes after the base use case. It is represented by a dashed arrow in the direction of the base use case with the notation <<extend>>.
 - **Include relationship:** The use case is mandatory and part of the base use case. It is represented by a dashed arrow in the direction of the included use case with the notation <<include>>.



Example

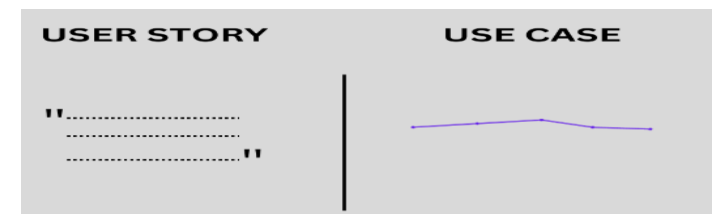
Use case
diagram of
Medical and
Care Services



Example

Use case
diagram of
Food
Resturant

Use Case vs User Story



Parameters	User Story	Use Case
Descriptions	The user story contains simplified and short descriptions.	A use case contains complete and lengthy descriptions.
Purpose	It is used to capture the requirements of a project in a <u>simpler</u> and more concise way.	It is designed to provide a <u>detailed</u> description of how a system should work.
Ease of Use	Users may read user stories with ease.	Describes in detail the interaction between users and the system.
Type of Interaction	It provides a single description.	It contains the sequence of interactions.
Management	It is manageable in one sprint.	It spans many sprints.
Time requirement	It is typically written in one or two sentences and is easy to read and understand.	It is usually much longer than User Stories, as they provide a more detailed description of how a system should work.

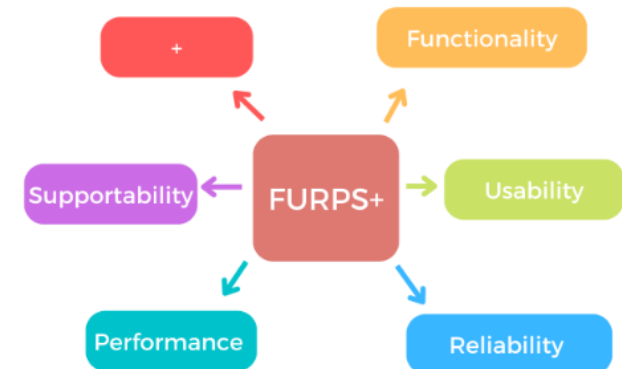
Requirements Engineering

- Goal:
 - Define and maintain requirements over time
- Activities:
 1. Requirements elicitation
(interviews, workshops, brainstormings, focus groups, ...)
 2. Requirements structuring
(requirements are annotated)
 3. User experience design
(meets user needs)
 4. Requirements validation

Requirements Structuring

- Goal:
 - Improving maintenance of requirements over time
- Tools:
 - Isolated and made identifiable (reason and manipulate each requirement more easily)
 - assigned a unique identifier
 - Organized and classified (e.g., FURPS+)
 - distinguishes between functional and nonfunctional requirements
 - Annotated (priority)
 - each requirement at least two properties:
 - **importance** for the customer
 - **difficulty to develop**, for instance, using values from 1 to 5

One method of categorizing functional and non-functional requirements is FURPS+



- **Annotated - Other attributes:**

- Traceability: define where a requirement originates from. Shows the relationships among requirements
 - This allows one to understand the impact of changes – make sure all components have similar quality and adequate testing.
- History log: traces how requirements have changed over time

Description
[ID] As a [user] I want to do [this] because of [that]

Attributes

Importance:	[IMPORTANCE]
Priority:	[PRIORITY]
Traceability:	[THIS REQUIREMENT RELATES TO ...]

Revision History

- [DATE] [AUTHOR] [DESCRIPTION]

A template for a requirement.

• User Experience Design

- Goal:

- Providing a coherent and satisfying experience on the different artifacts that constitute a software system, including its design, interface, interaction, and manuals

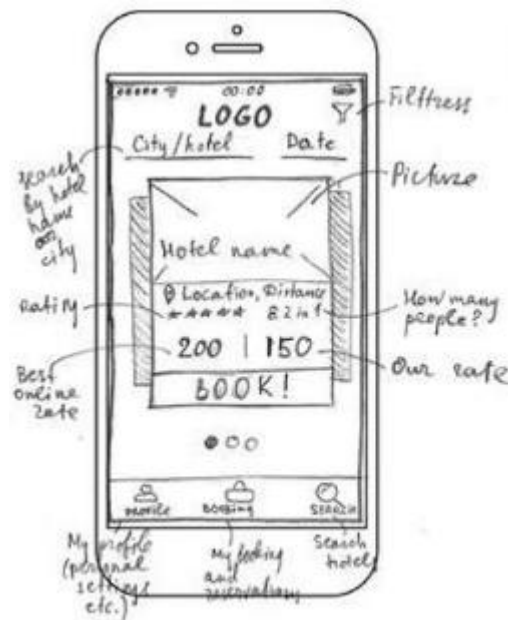
- Tools:

- **User-centered analysis**: understanding how users will interact with the system (focus groups, experiments, surveys)
- **User-centered design**: specifying how users will actually interact with the system (mock-ups)

UX design focuses on the end-users and their actual needs

Wireframe, Mockup, And Prototype

- A **wireframe** is a quick sketch of a product intended to convey its desired functionalities.
 - is rarely deployed as a testing material but helps build consensus and gather early feedback
- A **mockup** is a realistic design of a product designed to gather feedback on its visual elements.
- A **prototype** is an interactive simulation of a product designed to test the user experience.
 - test user experience before allowing the developers or engineers to begin their work



Wireframe



Mockup



Prototype

Requirements Validation

- Find (and address):
 - Inconsistencies (behave in contradictory ways)
 - * scenario 1: **R1. A; ...; Rn: not A**
 - * scenario 2: **R1. forall x. A(x); ...; Rn: not A(c)**
 - Incompleteness
 - * the behavior is not specified for certain cases and situations (often non-nominal situations)
 - * software engineers fill in the requirements' gaps by getting feedback from the stakeholders or by making explicit or implicit assumptions. Explicit assumptions can be validated during analysis, while implicit assumptions validation is carried to design and implementation.
 - Duplicates
 - * the same requirements is described twice (possibly in different ways)
 - * Combating code duplication to reduce software maintenance cost

Inconsistencies

Inconsistencies in requirements occur when different parts of the requirements behave in contradictory ways. For instance, one requirement might suggest a behavior that conflicts with another, leading to confusion during development or testing.

Scenario 1:

- R1:** "The system must allow users to log in."
- Rn:** "The system must prevent any user from logging in."

In this example, R1 and Rn contradict each other. One requirement says logging in is allowed, while the other says it is not allowed. This is clearly an inconsistency.

Incompleteness

Incompleteness arises when the behavior of the system is not specified for certain situations, especially those that fall outside the "nominal" (normal or expected) use cases.

Example:

- R1:** "The system must allow a user to place an order."
- R2:** "The system must display an error message if the user enters an invalid address."
- Missing Requirement:** What should happen if the user places an order but there are no available items in stock? This situation is not covered by the requirements.

Incompleteness can cause confusion during development and testing because edge cases or unusual situations aren't explicitly addressed in the documentation.

Duplicates

Duplicates occur when the same requirement is described multiple times, either in different ways or within different sections of the documentation. This redundancy can lead to confusion, miscommunication, and extra work when implementing the requirement.

Example:

- R1:** "The system must allow a user to view their transaction history."
- R2:** "The system must allow a user to check their past transactions."

While R1 and R2 may seem different, they essentially describe the same feature: allowing a user to access their transaction history. This redundancy is unnecessary and could lead to confusion about whether both are required or if the requirements were misunderstood.

Business Process Modeling and Re-engineering

Organizations and Software

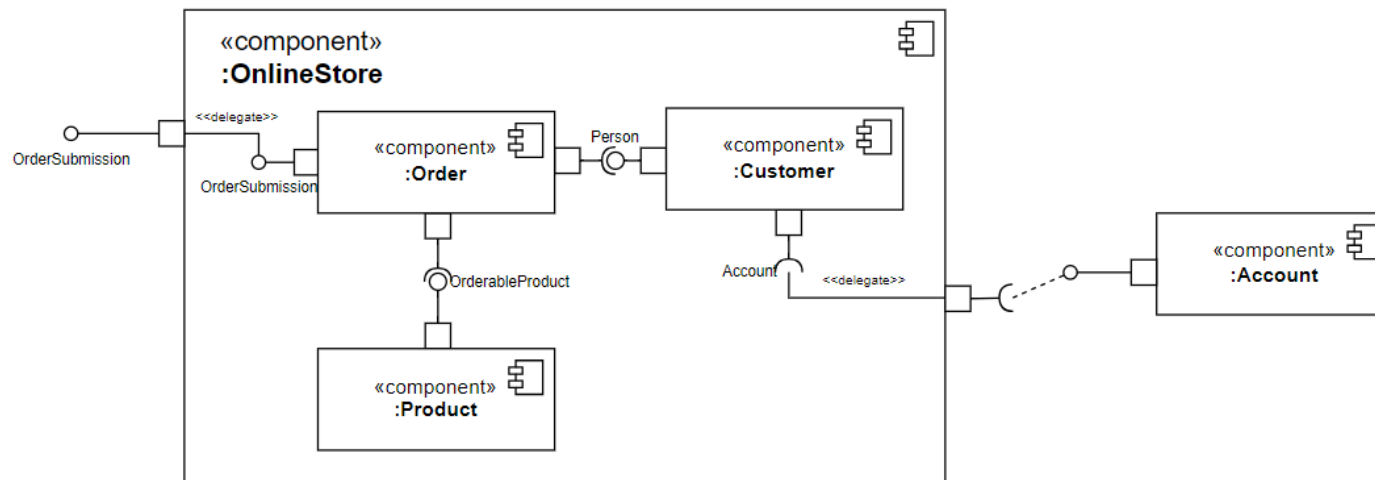
- Software has to be designed to fit an organization's operational structure
- However: software can also change the way in which an organization work
- Utilizing ERP systems:
 - (ERP) are systems that can automate and simplify the processes of an organization, integrating the data and the procedures of different business units.
 - understanding how work is carried out in an organization is often more **relevant than eliciting** the requirements of the system to be built, since an important part of the project work will focus on **mapping** the current procedures to accommodate those that supported by the **ERP** system.
- **Business process modeling** models the way in which an organization works
- **Business process re-engineering** plans the way in which an organization works, to make its operations more efficient – two steps:
 - “**as is**” analysis describes the organization before the introduction of a new system.
 - “**to be**” phase defines how the organization will change with the introduction of the new system

Business Process Modeling

- Articulated مفصل and complex معقد , it is sometimes planned and organized as an independent project
- Conducted with interviews, document analysis, shadowing
 - Shadow processes (العمليات غير المتتبعة) are business tasks and workflows that are typically not formally tracked or monitored by the organization. This can include manual processes, undocumented activities
- Information to collect:
 - **Organizational structure:** chain of responsibility [task-focused] and accountability [results-focused] - identify the changes that will have to be implemented in the organization to support the new processes
 - **Business processes:** how an organization carries out its procedures – flow diagrams sketched.
 - **Existing IT infrastructure:** what IT systems are currently used in an organization - hardware, systems, databases. Component and deployment diagrams are used.
 - **Business entities:** data produced and processed by the organization. Data is presented in class diagrams CRUD matrices.

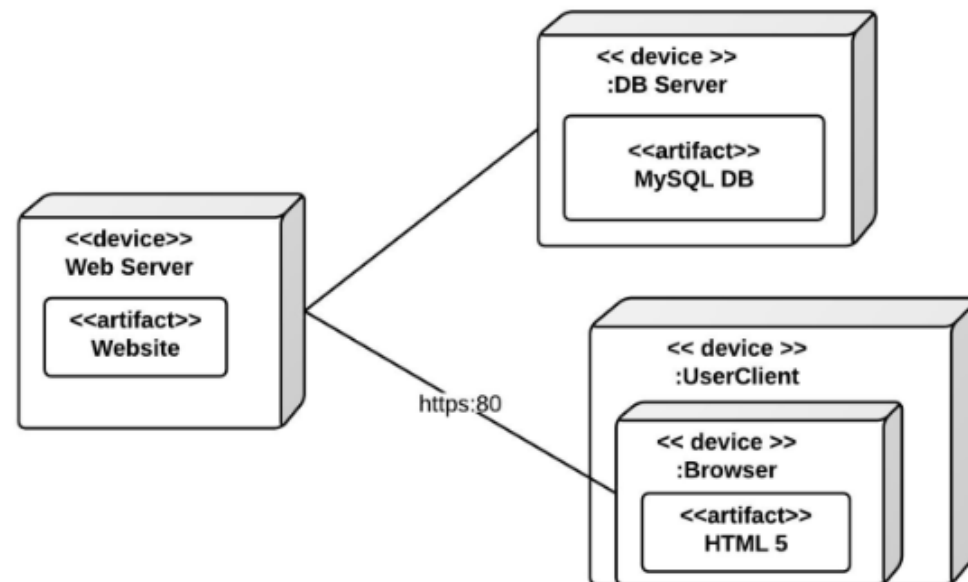
Component Diagram

- UML component diagrams are used to model the high-level software components and subsystems in service-oriented architectures
- They are related to **class** diagrams in that they also provide implementation details to developers, in this case by defining the **interfaces** through which the various components interact.
 - Unlike class diagrams, the detail of the internal data structure or methods within the component itself are hidden, showing only the interfaces used to interact with that component. Decoupling the internal workings of a component from external parts of the system.
- Component diagrams encourage developers to design modular components that can be reused both within a complex system, and in other projects.



Deployment Diagram

- The deployment diagram describes the physical deployment of information generated by the software program on hardware components. The information that the software generates is called an artifact.
- The three-dimensional boxes, known as **nodes**, represent the basic software or hardware elements.
- Deployment diagrams have several valuable applications.
 - Show which software elements are deployed by which hardware elements.
 - Provide a view of the hardware system's topology.



Component vs Deployment Diagram

Component diagrams focus on the high-level structure and organization of software components within a system, while deployment diagrams concentrate on how these components are physically deployed on hardware or nodes

Aspect	Component Diagram	Deployment Diagram
Purpose	Represent the high-level software structure	Model the physical deployment of software components
Focus	Logical organization and relationships of components	Physical deployment on hardware and nodes
Main Elements	– Components (e.g., classes, packages, subsystems)	– Nodes (e.g., servers, workstations)
	– Interfaces (e.g., contracts, APIs)	– Artifacts (e.g., files, databases)
	– Connectors (e.g., associations, dependencies)	– Associations (indicating deployment relationships)
Use Cases	– Design phase: System architecture and structure	– Implementation phase: Deployment planning
	– Illustrate component relationships and interfaces	– Visualize component distribution on hardware
Abstraction Level	High-level abstraction	Low-level abstraction
Notation	Components, interfaces, connectors	Nodes, artifacts, associations
Relationships Represented	Dependencies, associations, aggregations, etc.	Deployment associations, mapping of components to nodes
Example Scenario	Representing software modules and their interactions in a banking application	Visualizing how web server software components are deployed on physical servers

CRUD Matrix Example

A CRUD matrix is a table that outlines the different types of CRUD operations that can be performed on different types of data in a system. It can be used as a reference guide to help developers understand which operations are allowed or required for different types of data and to help them design and implement the data storage and retrieval systems in their applications

CRUD (Create, Read, Update, Delete) matrix represents the four basic functions of persistent storage in a database, a system, or an application. The matrix is typically used to represent the permissions or capabilities of different roles in an application.

	Customer	Employee	Manager
Account	R, U, C	R, D, C	R, U, D, C
Order	C, R, U	C, R, U, D	D, R
Product	R	C, R, U, D	R, D

Thank You